

Diffusion Models: A Comprehensive Survey of Methods and Applications, Part 1

By Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng
Xu, Yue Zhao, Wentao Zhang, Bin Cui, Ming-Hsuan Yang

Presented by: Evgenii Samutichev

January 23, 2025

Outline

1 Introduction (Chapter 1)

2 Three predominant mathematical formulations (Chapter 2)

- Denoising diffusion probabilistic models (DDPMs)
 - Forward process
 - Reverse process
 - Training DDPM with VLB loss
- Score-based Generative Models (SGMs)
 - Training SGM with denoising score matching
 - Sampling SGMs with annealed Langevin dynamics (ALD)
- Score SDEs
 - Training of and sampling from score SDEs

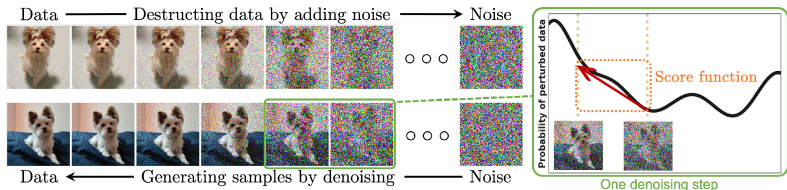
3 Directions of improvement (Chapters 3-4)

- Efficient sampling
 - Learning-free sampling
 - Learning-based sampling
- Improved likelihood (introduction)
 - Reverse variance learning

Introduction (Chapter 1)

Introduction (Chapter 1)

- New state-of-the-art family of deep generative models, which beat GANs on several tasks, including image synthesis (e.g. Stable Diffusion, DALL-E)
- The body of research on diffusion models has grown significantly, many interesting approaches and mathematics.
- **Key idea:** progressively perturbing data with intensifying random noise, while learning how to remove noise to generate new data samples.



Three predominant mathematical formulations (Chapter 2)

DDPM key ideas

- We first consider **Denoising diffusion probabilistic models (DDPMs)** formulation.
- Two Markov chains: **forward** that introduces noise, **reverse** that converts noise back to data.
- Forward is typically hand-designed, reverse has to be learned
- Starting from an unknown data distribution $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, forward process $\mathbf{x}_1, \dots, \mathbf{x}_T$ with transition kernel $q(\mathbf{x}_t | \mathbf{x}_{t-1})$.
- Starting from some known noise distribution $\mathbf{x}_T \sim p(\mathbf{x}_T)$, reverse process with transition kernel $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ parametrized by deep neural networks, should generate $\mathbf{x}_{T-1}, \dots, \mathbf{x}_0$.

Forward process

- Joint distribution (chain rule of probability + Markov property)

$$q(\mathbf{x}_1, \dots, \mathbf{x}_T \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \dots, \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \quad (1)$$

- The most common choice is Gaussian

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad \beta_t \in (0, 1) \quad (2)$$

- **Sohl-Dickstein formula**

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \quad (3)$$

where $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=0}^t \alpha_s$

- Then with the right choice of $\beta_t \in (0, 1)$, we have

$$q(\mathbf{x}_t \mid \mathbf{x}_0) \rightarrow \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ as } t \rightarrow \infty$$

Sohl-Dickstein formula, a sketch of the proof (1 / 2)

- For two independent Gaussian kernels of the form $\mathcal{N}(\mathbf{x}; \mu_{\mathbf{x}}, \beta_{\mathbf{x}}\mathbf{I})$ and $\mathcal{N}(\mathbf{y}; \mu_{\mathbf{y}}, \beta_{\mathbf{y}}\mathbf{I})$, their product is another Gaussian kernel but for $\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}$ with mean $\begin{bmatrix} \mu_{\mathbf{x}} \\ \mu_{\mathbf{y}} \end{bmatrix}$ and covariance matrix $\begin{bmatrix} \beta_{\mathbf{x}}\mathbf{I} & 0 \\ 0 & \beta_{\mathbf{y}}\mathbf{I} \end{bmatrix}$.

$$q(\mathbf{x}_1, \dots, \mathbf{x}_T \mid \mathbf{x}_0) = \mathcal{N} \left(\mathbf{x}_1, \dots, \mathbf{x}_T; \begin{bmatrix} \sqrt{\alpha_1}\mathbf{x}_0 \\ \dots \\ \sqrt{\alpha_T}\mathbf{x}_{T-1} \end{bmatrix}, \begin{bmatrix} \beta_1\mathbf{I} & \dots & 0 \\ 0 & \dots & 0 \\ 0 & \dots & \beta_T\mathbf{I} \end{bmatrix} \right)$$

- Focusing only on $\mathbf{x}_1$ under the exponent

$$-\frac{1}{2\beta_1}(\mathbf{x}_1 - \sqrt{\alpha_1}\mathbf{x}_0)^T(\mathbf{x}_1 - \sqrt{\alpha_1}\mathbf{x}_0) - \frac{1}{2\beta_2}(\mathbf{x}_2 - \sqrt{\alpha_2}\mathbf{x}_1)^T(\mathbf{x}_2 - \sqrt{\alpha_2}\mathbf{x}_1)$$

Sohl-Dickstein formula, a sketch of the proof (2 / 2)

- Changing the variable $\mathbf{x}_1 \rightarrow \frac{1}{\sqrt{\alpha_2}}(\mathbf{x}_2 - \mathbf{x}_1)$, we get

$$-\frac{1}{2\beta_2}\mathbf{x}_1^T\mathbf{x}_1 - \frac{1}{2\beta_1\alpha_2}(\mathbf{x}_1 - (\mathbf{x}_2 - \sqrt{\alpha_1\alpha_2}\mathbf{x}_0))^T(\mathbf{x}_1 - (\mathbf{x}_2 - \sqrt{\alpha_1\alpha_2}\mathbf{x}_0))$$

- Then, use the formula

$$\int_{\mathbf{R}^n} \mathcal{N}(\mathbf{x}; \mathbf{0}, \Gamma) \mathcal{N}(\mathbf{x}; \mu, \Sigma) d\mathbf{x} = \frac{\exp\left(-\frac{1}{2}\mu^T(\Gamma(\Sigma^{-1} + \Gamma^{-1})\Sigma)^{-1}\mu\right)}{\sqrt{(2\pi)^n |\Sigma| |\Gamma| |\Sigma^{-1} + \Gamma^{-1}|}} \quad (4)$$

- to get under the exponent

$$-\frac{1}{2(1 - \alpha_1\alpha_2)}(\mathbf{x}_2 - \sqrt{\alpha_1\alpha_2}\mathbf{x}_0)^T(\mathbf{x}_2 - \sqrt{\alpha_1\alpha_2}\mathbf{x}_0)$$

Reverse process

- Learnable transition kernel, also Gaussian

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t)) \quad (5)$$

with $\mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t)$ represented by DNNs.

- Joint distribution

$$p_{\theta}(\mathbf{x}_0, \dots, \mathbf{x}_T) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \quad (6)$$

- └ Three predominant mathematical formulations (Chapter 2)

- └ Denoising diffusion probabilistic models (DDPMs)

Training DDPM with VLB loss

- We want to match the reverse process with the forward process in distribution, thus we start with KL-divergence as a loss

$$\text{KL}(q(\mathbf{x}_0, \dots, \mathbf{x}_T) \parallel p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)) = -\mathbb{E}_{q(\mathbf{x}_0, \dots, \mathbf{x}_T)} \left[\log \frac{p(\mathbf{x}_0, \dots, \mathbf{x}_T)}{q(\mathbf{x}_0, \dots, \mathbf{x}_T)} \right].$$

- Can be simplified to get the **Variational Lower Bound (VLB)** of the log-likelihood of the data $\mathbf{x}_0$

$$\begin{aligned} \text{KL}(q(\mathbf{x}_0, \dots, \mathbf{x}_T) \parallel p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)) &= \\ &= -\underbrace{\mathbb{E}_{q(\mathbf{x}_0, \dots, \mathbf{x}_T)} \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right]}_{L_{\text{VLB}}(\mathbf{x}_0)} + \text{const} \end{aligned}$$

More on VLB

- To better understand where VLB comes from, we first fix $\mathbf{x}_0$ and write by Jensen's inequality

$$-\log p_{\theta}(\mathbf{x}_0) \leq \mathbb{E}_{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} \left[-\log \frac{p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)}{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} \right] \quad (7)$$

- Then, we take expectation w.r.t. $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ of both sides, and multiply by -1 resulting in

$$\log p_{\theta}(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} \left[\log \frac{p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)}{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} \right] = L_{\text{VLB}}(\mathbf{x}_0)$$

- We can then use expressions (1) and (6) to get that connection with KL-divergence (maximizing one minimizes the other)

SGM key ideas

- Next, consider **Score-based Generative Models (SGMs)** formulation.
- Perturb data with a sequence of intensifying Gaussian noise and jointly estimate the score functions for all noisy data distributions by training a deep neural network model conditioned on noise levels (called a noiseconditional score network, NCSN).
- Given a PDF $p(\mathbf{x})$, the **Stein score function** is $\nabla_{\mathbf{x}} \log p(\mathbf{x})$.
- Consider a sequence of noise levels $0 < \sigma_1 < \sigma_2 < \dots < \sigma_t < \dots < \sigma_T$ and let $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \mathbf{x}_0, \sigma_t^2 \mathbf{I})$, then $q(\mathbf{x}_t) = \int q(\mathbf{x}_t | \mathbf{x}_0) q(\mathbf{x}_0) d\mathbf{x}_0$.
- Want to estimate scores $\mathbf{s}_\theta(\mathbf{x}, t) = \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$ with DNN to then use as directions in the sampling algorithm based of Langevin dynamics.

Training SGM with denoising score matching (1 / 2)

- *Ho et al. (2020)* paper introduces the following loss function with weights $\lambda(t)$ for DDPM

$$\mathbb{E}_{t \sim \mathcal{U}(\{1, \dots, T\}), \mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\lambda(t) \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] \quad (8)$$

where $\epsilon_\theta(\mathbf{x}_t, t)$ predicts added noise and we compute $\mathbf{x}_t$ based on Sohl-Dickstein formula

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (9)$$

- This one is equivalent to what we had before under a certain choice of $\lambda(t)$.

Training SGM with denoising score matching (2 / 2)

- Now, for SGM, we introduce and transform a similar type of loss, called **denoising score matching**. For now assume we are taking expectation over

$$t \sim \mathcal{U}(\{1, \dots, T\}), \mathbf{x}_0 \sim q(\mathbf{x}_0), \mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} [\lambda(t) \sigma_t^2 \|\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) - \mathbf{s}_\theta(\mathbf{x}_t, t)\|^2] \quad (10)$$

$$\stackrel{*}{=} \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} [\lambda(t) \sigma_t^2 \|\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) - \mathbf{s}_\theta(\mathbf{x}_t, t)\|^2] + \text{const} \quad (11)$$

$$= \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} \left[\lambda(t) \left\| -\frac{\mathbf{x}_t - \mathbf{x}_0}{\sigma_t} - \sigma_t \mathbf{s}_\theta(\mathbf{x}_t, t) \right\|^2 \right] + \text{const} \quad (12)$$

$$= \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\lambda(t) \|\epsilon + \sigma_t \mathbf{s}_\theta(\mathbf{x}_t, t)\|^2] + \text{const} \quad (13)$$

★ is known as an equivalence between **explicit score matching (ESM)** and DSM, and we have used that $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t \epsilon$

- Compare with DDPM loss with $\epsilon_\theta(\mathbf{x}_t, t) = -\sigma_t \mathbf{s}_\theta(\mathbf{x}_t, t)$.

Sampling SGMs with annealed Langevin dynamics (ALD)

- Parameters: N – number of iterations per step size, $s_t > 0$ – step size, initialize with $\mathbf{x}_T^{(N)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- Then, for $t = T, T - 1, \dots, 1$, initialize $\mathbf{x}_t^{(0)} = \mathbf{x}_{t-1}^{(N)}$ and then for $i = 0, 1, \dots, N - 1$:

$$\epsilon^{(i)} \leftarrow \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (14)$$

$$\mathbf{x}_t^{(i+1)} \leftarrow \mathbf{x}_t^{(i)} + \frac{1}{2}s_t \mathbf{s}_\theta(\mathbf{x}_t^{(i)}, t) + \sqrt{s_t} \epsilon^{(i)}. \quad (15)$$

- The theory of Langevin Monte Carlo guarantees that as $s_t \rightarrow 0$ and $N \rightarrow \infty$, $\mathbf{x}_0^{(N)}$ becomes a valid sample from the data distribution $q(\mathbf{x}_0)$.

Score SDEs key ideas (1 / 2)

- Continuous analogue of DDPMs and SGMs, to which they relate as discretizations, e.g. for DDPM with $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ i.i.d.
 $t = 0, \Delta t, 2\Delta t, \dots, M\Delta t = T$

$$\mathbf{x}_{t+\Delta t} = \sqrt{1 - \beta_t \Delta t} \mathbf{x}_t + \sqrt{\beta_t \Delta t} \epsilon_t \quad (16)$$

$$\Delta \mathbf{x}_t = (\sqrt{1 - \beta_t \Delta t} - 1) \mathbf{x}_t + \sqrt{\beta_t \Delta t} \epsilon_t \quad (17)$$

$$\Delta \mathbf{x}_t = \frac{\sqrt{1 - \beta_t \Delta t} - 1}{\Delta t} \mathbf{x}_t \Delta t + \sqrt{\beta_t} \sqrt{\Delta t} \epsilon_t \quad (18)$$

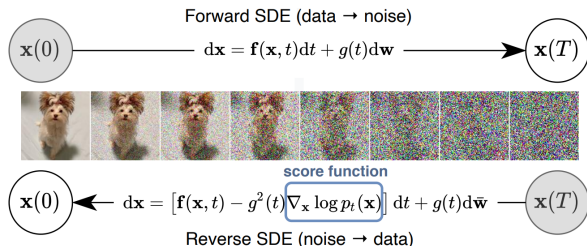
then as $\Delta t \rightarrow 0$ we have by using Taylor expansion

$\sqrt{1+z} = 1 + \frac{1}{2}z + \dots$ and knowing that $\Delta \mathbf{w}_t, \sqrt{\Delta t} \epsilon_t \sim \mathcal{N}(\mathbf{0}, \Delta t \mathbf{I})$

$$d\mathbf{x}_t = -\frac{1}{2}\beta_t \mathbf{x}_t dt + \sqrt{\beta_t} d\mathbf{w}_t. \quad (19)$$

Score SDEs key ideas (2 / 2)

- Generally the following picture, except replace $p_t(\mathbf{x})$ by $q_t(\mathbf{x})$ which is the distribution of $\mathbf{x}_t$ in the forward process



- Probability flow ODE** introduced by *Song et. al (2020)* has the same marginals as the reverse SDE

$$d\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log q_t(\mathbf{x}_t) \right] dt \quad (20)$$

Simplified derivation

- Starting with a forward SDE

$$d\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t)dt + g(t)d\mathbf{w}_t, \quad (21)$$

we can write Kolmogorov Forward Equation for its marginal densities

$$\frac{\partial q_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}} \cdot \left[\mathbf{f}(\mathbf{x}, t)q_t(\mathbf{x}) - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}}q_t(\mathbf{x}) \right] \quad (22)$$

- Using $\nabla_{\mathbf{x}} \log q_t(\mathbf{x}) = \frac{\nabla_{\mathbf{x}}q_t(\mathbf{x})}{q_t(\mathbf{x})}$ we can rewrite

$$\frac{\partial q_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}} \cdot \left[(\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}} \log q_t(\mathbf{x}))q_t(\mathbf{x}) \right] \quad (23)$$

Training of and sampling from score SDEs

- Once we know $q_t(\mathbf{x})$, sampling is straightforward either from the SDE with annealed Langevin dynamics or from the probability flow ODE with some numerical ODE solver
- We don't know $q_t(\mathbf{x})$ and obtaining it directly from KFE isn't really feasible, hence score matching again for $t \sim \mathcal{U}[0, T], \mathbf{x}_0 \sim q(\mathbf{x}_0), \mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0)$

$$\mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} \left[\lambda(t) \|\mathbf{s}_\theta(\mathbf{x}, t) - \nabla_{\mathbf{x}_t} \log q_{0t}(\mathbf{x}_t | \mathbf{x}_0)\|^2 \right] \quad (24)$$

Directions of improvement (Chapters 3-4)

Directions of improvement

Subsequent research on diffusion models focuses on improving these classical approaches (DDPMs, SGMs, and Score SDEs) from three major directions:

- 1 faster and more efficient sampling;
- 2 more accurate likelihood and density estimation;
- 3 handling data with special structures (such as permutation invariance, manifold structures, and discrete data).

Efficient sampling

- The main goal is in improving either SDE or ODE solvers, decreasing the number of discretized time steps, while minimizing the error.
- **Learning-free** sampling is a paradigm of using handcrafted steps, while **learning-based** sampling is based on optimizing certain learning objectives.

Learning-free sampling

Some SDE improvements

- **Predictor-corrector for SDE**: coarse SDE solver (predictor) and iterative MCMC procedure to nudge it to the correct marginal (corrector)
- **Adaptive step size**: from *Jolicoeur-Martineau et al. (2021)* there is an SDE solver with adaptive step sizes for faster generation, based on comparing a high-order and a low-order SDE solvers

Some ODE improvements

- **Diffusion Exponential Integrator Sampler** utilizes semi-linear structure of probability flow ODE to get an edge over the general-purpose methods, for DDPM

$$d\mathbf{x}_t = \left[-\frac{1}{2}\beta_t \mathbf{x}_t - \frac{1}{2}\beta_t \nabla_{\mathbf{x}} \log q_t(\mathbf{x}_t) \right] dt \quad (25)$$

Learning-based sampling

- **Optimized discretization:** given a pre-trained model, we are looking for the best K time-steps, this optimization becomes feasible with use of dynamic programming
- **Truncated diffusion:** truncating forward and reverse diffusion processes, starting denoising earlier and generating initial samples with some pre-trained generative model

Improved likelihood (introduction)

- VLB of the log-likelihood is not a tight bound in many cases, so optimizing it leads to suboptimal results
- We want to achieve likelihood maximization and there are three methods
 - 1 noise schedule optimization;
 - 2 **reverse variance learning**;
 - 3 exact log-likelihood evaluation.

Reverse variance learning

- Even though we introduced $\Sigma_\theta(\mathbf{x}_t, t)$ in DDPM formulation, in practice people used $\beta\mathbf{I}$ in place of it (no learning)
- The idea of the **reverse variance learning** is to write a representation which will allow to efficiently learn the reverse variance

$$\Sigma_\theta = \exp\left(\theta \cdot \log \beta_t + (1 - \theta) \cdot \log \tilde{\beta}_t\right), \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \quad (26)$$

- Jointly train β_t and θ to maximize VLB.
- In *Analytic-DPM* they found an optimal reverse variance if one knows the score functions.